

ĐỀ TÀI: XÂY DỰNG HỆ THỐNG QUẢN LÝ THƯ VIỆN TRỰC TUYẾN TÍCH HỢP AI HỖ TRỢ TÌM KIẾM VÀ GỢI Ý SÁCH.



Giảng viên hướng dẫn: TS. Trương Tuấn Anh
CN. Dương Huỳnh Anh Đức

Sinh viên thực hiện: Võ Quang Thắng – 2213214



MÃ DỰ ÁN: HK253-DATN-076

ĐẶT VẤN ĐỀ & BỐI CẢNH



BỐI CẢNH

Bạn đọc khó tìm tài liệu nếu không nhớ đúng tên sách hoặc từ khóa.



THỦ THƯ

Thủ thư mất nhiều thời gian nhập liệu, xử lý yêu cầu, nhắc hạn và đối soát phí công kênh.



QUẢN LÝ THỦ CÔNG

Dữ liệu thư viện phân tán, khó thống kê và khó ra quyết định.



KHAI THÁC PDF

Tài liệu PDF có nội dung phong phú nhưng chưa được khai thác bằng tìm kiếm ngữ nghĩa.



DANH MỤC THỂ LOẠI

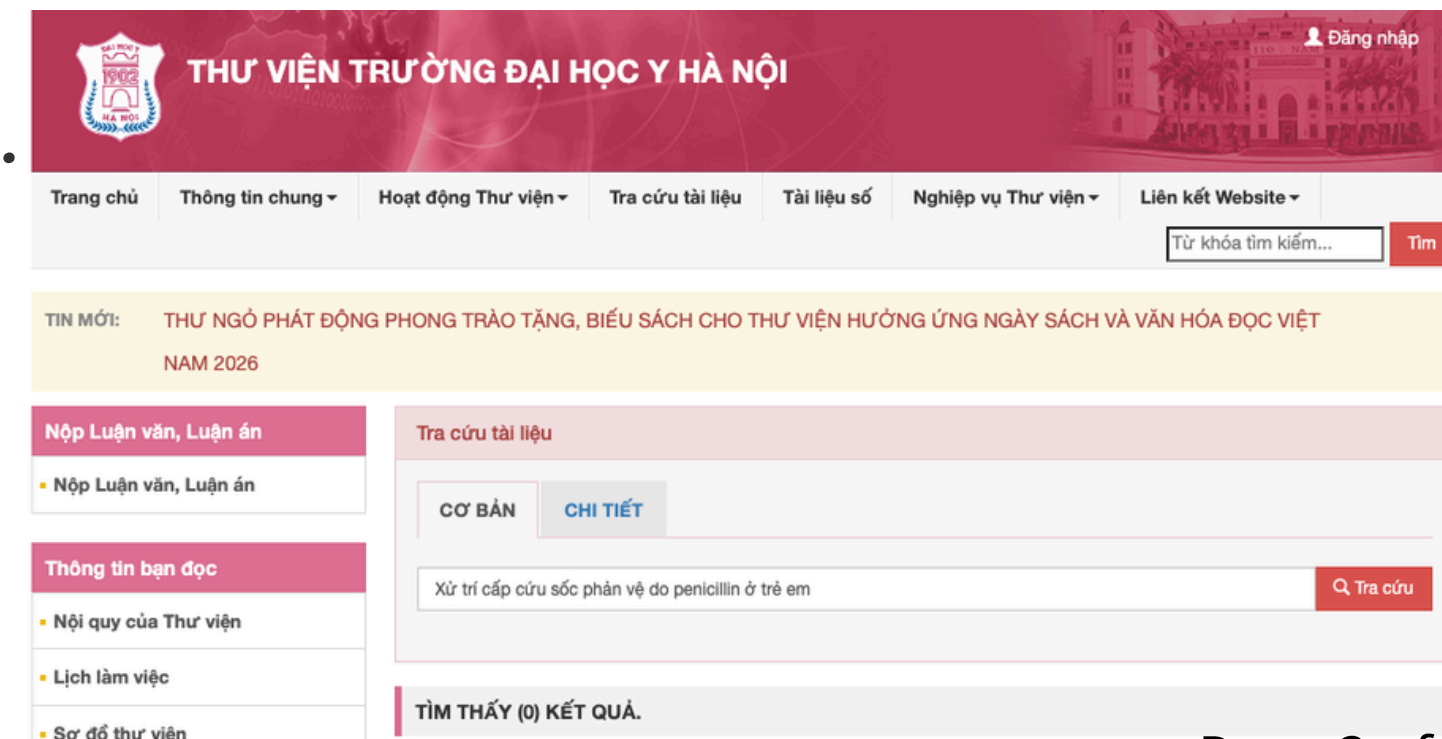


Lưới



Danh sách

Trang: 1



MỘT SỐ CÔNG TRÌNH LIÊN QUAN

OPAC-HCMUT-VNU

Ưu điểm:

- Ổn định
- Đáp ứng nghiệp vụ tra cứu và mượn– trả cơ bản.

Nhược điểm:

- Tìm kiếm từ khóa
- Không gợi ý sách
- Không phân tích hành vi
- Giao diện chưa tối ưu mobile



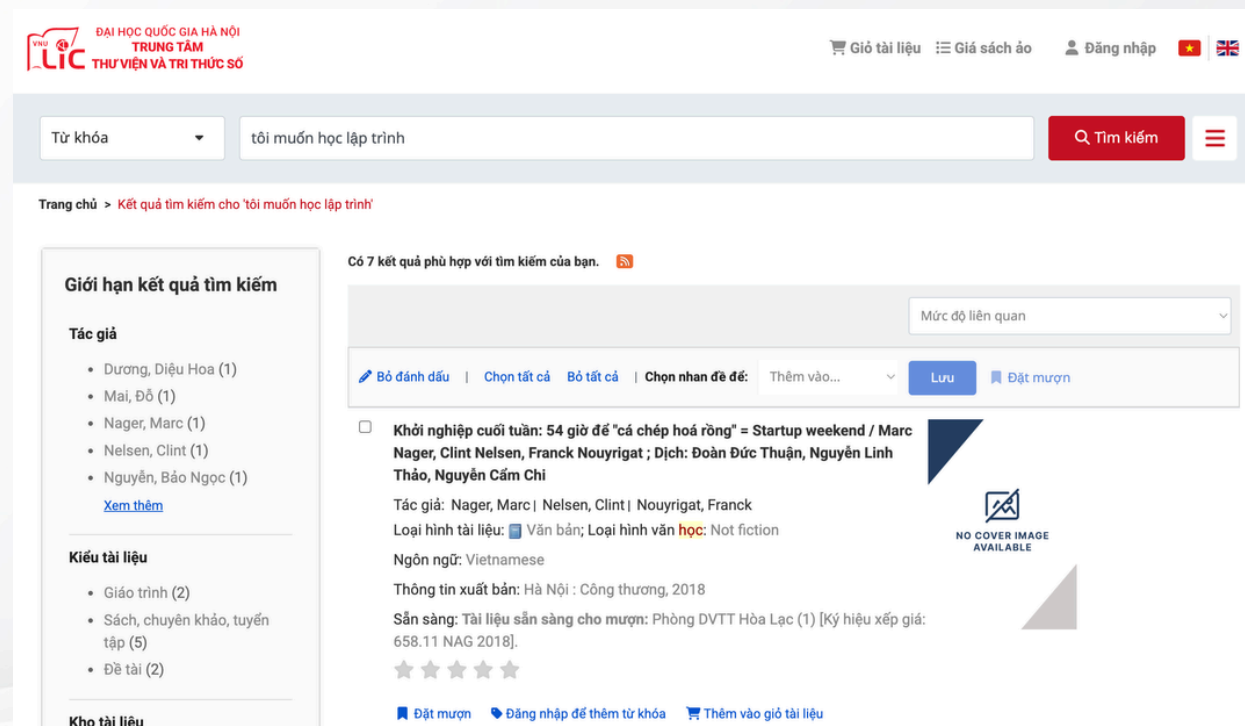
KOHA (MÃ NGUỒN MỞ)

Ưu điểm:

- Đầy đủ chức năng quản lý
- Cộng đồng lớn, dễ triển khai.

Nhược điểm:

- Giao diện cũ
- Không tích hợp AI/NLP
- Tìm kiếm không ngữ nghĩa
- Khó mở rộng tính năng thông minh.



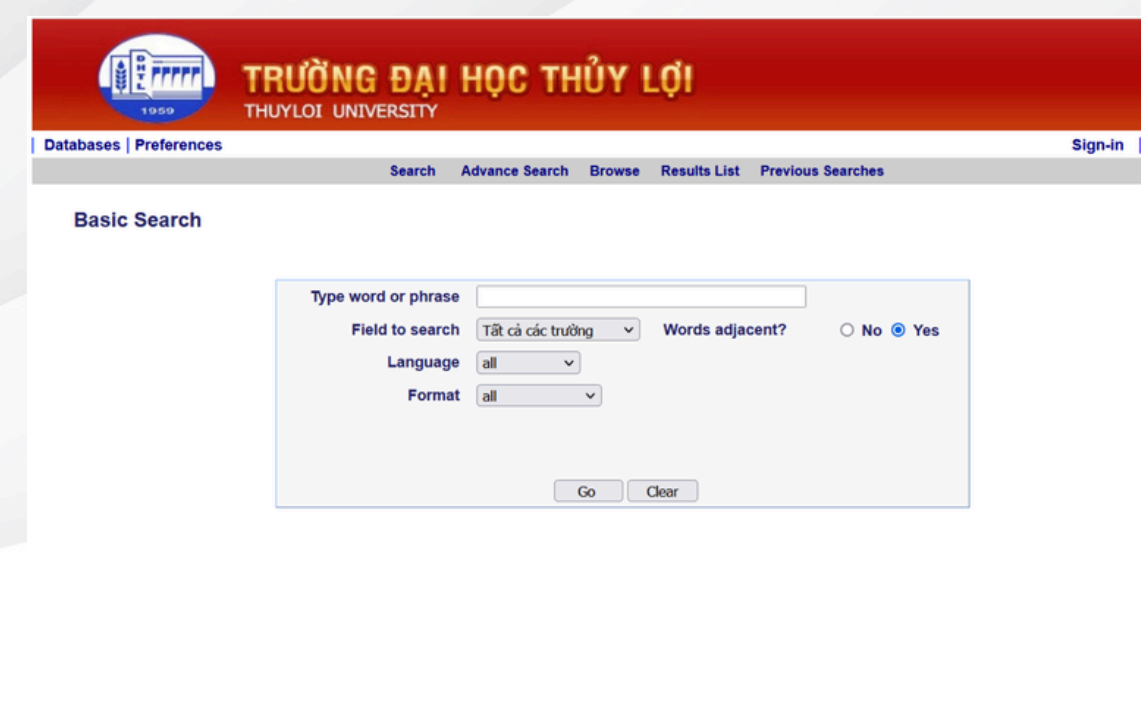
ALEPH (EX LIBRIS)

Ưu điểm:

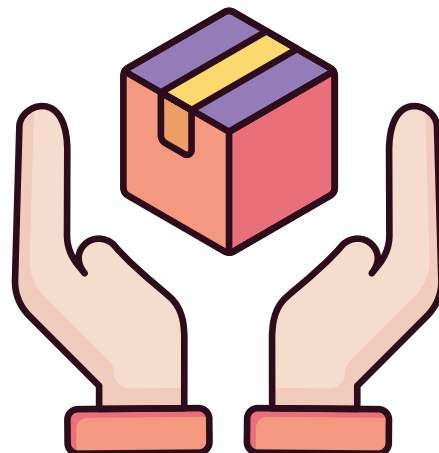
- Quy mô lớn, tích hợp Nhiều CSDL học thuật
- Ổn định cao.

Nhược điểm:

- Chi phí rất cao
- Phụ thuộc nhà cung cấp
- Khó tùy biến
- Không phù hợp thư viện quy mô nhỏ.



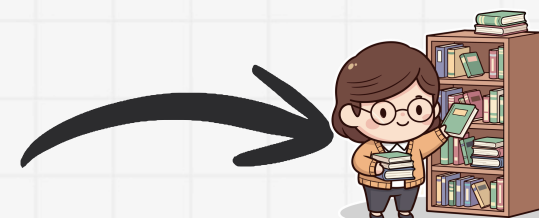
GIẢI PHÁP ĐỀ XUẤT

MỘT NỀN TẢNG WEB NHIỀU VAI TRÒ	TÍCH HỢP AI NHƯ MỘT DỊCH VỤ VỆ TINH	TRIỂN KHAI PRODUCTION TRÊN DOMAIN	SẢN PHẨM
<i>Đối tượng sử dụng hệ thống là những ai?</i> • Khách truy cập • Bạn đọc • Thủ thư • Quản trị viên 	<i>Các tính năng AI cốt lõi cần giải quyết là gì?</i> • Tìm kiếm ngữ nghĩa • Hệ thống gợi ý • Trích xuất văn bản • Tự động làm giàu dữ liệu 	<i>Sản phẩm thực tế sẽ được vận hành ở đâu?</i> • https://library74.uk hoặc https://www.library74.uk 	<i>Kết quả của sản phẩm là gì?</i> • Mã nguồn hoàn chỉnh của nền tảng Web. • Hệ thống API kết nối các dịch vụ AI (Tìm kiếm ngữ nghĩa, hệ thống gợi ý). • Sản phẩm hoàn thiện chạy thực tế trên domain library74.uk. 

LỢI ÍCH



Bạn đọc:
Tìm sách nhanh
hơn, tự theo dõi
mượn/trả/phí



Thủ thư:
Giảm nhập liệu và
thao tác lặp



Quản trị viên:
Kiểm soát tài khoản,
cấu hình và chính sách



Nhà trường:
Có dữ liệu tập trung
để ra quyết định

HỆ THỐNG

Search, semantic search,
dashboard, reservation, fines,
wishlist

Publication management, ISBN
autofill, PDF upload, AI
processing, circulation

Admin user/librarian
management, policy, audit logs

Reports, dashboard, statistics,
search/borrow interaction data

LỢI ÍCH THỰC TẾ CỦA HỆ THỐNG

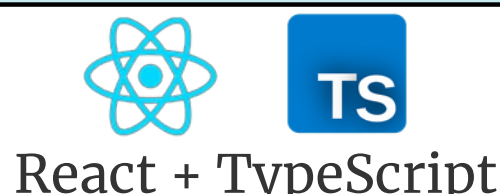
SO SÁNH KIẾN TRÚC VÀ LÝ DO CHỌN

PHƯƠNG ÁN	ƯU ĐIỂM	NHUỢC ĐIỂM	KẾT LUẬN
Monolith một khối	Đơn giản, dễ chạy ban đầu	Dễ rối khi nghiệp vụ lớn, khó tách AI Python	Không phù hợp vì hệ thống có nhiều phạm vi ngữ cảnh biên và AI
Microservices toàn phần	Scale độc lập, tách service rõ	Phức tạp vận hành, cần Định tuyến và Giám sát dịch vụ, tracing, CI/CD mạnh	Quá nặng cho đồ án và quy mô hiện tại
Modular monolith + dịch vụ AI vệ tinh	Backend nghiệp vụ ổn định, module rõ, AI tách riêng để tận dụng Python	Một số module backend chưa mở rộng độc lập	Phù hợp nhất với quy mô, dễ kiểm thử, dễ triển khai

- Không chọn monolith một khối vì hệ thống có nhiều nghiệp vụ, để lâu sẽ khó bảo trì.
- Không chọn microservices toàn phần vì quá nặng cho đồ án và làm phức tạp giao dịch, triển khai, giám sát.
- Chọn modular monolith vì dữ liệu thư viện liên kết chặt: người dùng, sách, bản sao, giao dịch, đặt trước và phí phạt.
- AI được tách thành service riêng vì khác ngôn ngữ, workload và hệ sinh thái xử lý.



FRONTEND: SO SÁNH CÔNG NGHỆ VÀ QUYẾT ĐỊNH

Phương án	Ưu điểm	Nhược điểm	Kết luận
	<i>Component và hook linh hoạt, phù hợp nhiều giao diện theo vai trò; hệ sinh thái định tuyến, kiểm thử và thông báo phong phú; TypeScript kiểm soát kiểu dữ liệu</i>	React chỉ tập trung vào giao diện, nhóm phải tự thống nhất cách quản lý trạng thái, gọi API và tổ chức mã nguồn	<i>Chọn vì Library74 cần khả năng tái sử dụng và linh hoạt, nhưng chưa cần cấu trúc bắt buộc của một framework đầy đủ</i>
	<i>Component một tệp dễ đọc; hệ thống phản ứng và Composition API tích hợp tốt; dễ tiếp cận</i>	Vẫn cần lựa chọn thêm thư viện cho các nhu cầu ngoài lõi; không tạo lợi thế kỹ thuật quyết định so với React trong phạm vi đề tài	<i>Phù hợp, nhưng không mang lại lợi ích đủ khác biệt để ưu tiên hơn React</i>
	<i>Framework đầy đủ, có sẵn định tuyến, biểu mẫu, HTTP client và dependency injection; cấu trúc thống nhất</i>	Nhiều quy ước và thành phần phải tuân theo; tăng độ phức tạp ban đầu so với nhu cầu của nhóm nhỏ	Phù hợp hệ thống cần chuẩn hóa rất chặt, nhưng nặng hơn phạm vi hiện tại

- Cả ba phương án đều xây dựng được giao diện nhiều vai trò; khác biệt nằm ở mức độ linh hoạt và số quy ước framework áp đặt.
- Angular cung cấp cấu trúc đầy đủ, Vue dễ tiếp cận, còn React cho phép em chọn đúng các thành phần cần thiết cho phạm vi Library74.
- => chọn React kết hợp TypeScript và chấp nhận trách nhiệm tự thống nhất kiến trúc giao diện.

BACKEND: SO SÁNH CÔNG NGHỆ VÀ QUYẾT ĐỊNH CHỌN

PHƯƠNG ÁN	ƯU ĐIỂM	NHƯỢC ĐIỂM	KẾT LUẬN
 Spring Boot	<i>@Transactional quản lý ranh giới giao dịch; Spring Security hỗ trợ xác thực và phân quyền; phù hợp tổ chức nghiệp vụ theo module</i>	Cấu trúc và cấu hình ban đầu nhiều; cần kỹ luật thiết kế và hiểu hệ sinh thái Java/Spring	<i>Chọn vì nghiệp vụ mượn, trả, đặt trước, phí phạt và thanh toán cần giao dịch và bảo mật được quản lý thống nhất</i>
 NestJS	<i>Module, controller, provider và dependency injection rõ ràng; dùng TypeScript đồng nhất với giao diện</i>	Phải lựa chọn ORM và tự thống nhất cách quản lý giao dịch, bảo mật, tác vụ nền giữa nhiều thư viện	<i>Phù hợp nếu ưu tiên TypeScript toàn hệ thống, nhưng tăng số quyết định tích hợp em phải kiểm soát</i>
 Django	<i>ORM, xác thực và trang quản trị tích hợp sẵn; phát triển chức năng quản trị nhanh</i>	Cách tổ chức chịu ảnh hưởng mạnh từ ORM và mô hình app; API thường cần kết hợp thêm Django REST Framework	Phù hợp khi ưu tiên phát triển nhanh bằng Python, nhưng không khớp bằng Spring Boot với cách tổ chức nghiệp vụ đã chọn

- Nghiệp vụ thư viện có nhiều thay đổi dữ liệu phải thành công hoặc thất bại cùng nhau, nên giao dịch là yếu tố quyết định.
- NestJS linh hoạt nhưng yêu cầu em chọn thêm nhiều thành phần; Django phát triển nhanh nhờ chức năng tích hợp; Spring Boot quản lý giao dịch và bảo mật thống nhất hơn với cách tổ chức em đã chọn.
- Spring Boot được chọn vì cơ chế giao dịch và bảo mật của nó khớp trực tiếp với rủi ro nghiệp vụ Library74 cần kiểm soát.

AI SERVICE: TÁCH RIÊNG ĐỂ XỬ LÝ WORKLOAD AI

PHƯƠNG ÁN	ƯU ĐIỂM	NHƯỢC ĐIỂM	KẾT LUẬN
Đặt AI trong Spring Boot	<i>Ít dịch vụ phải vận hành; nghiệp vụ và AI giao tiếp trực tiếp trong cùng hệ thống</i>	Khó dùng trực tiếp hệ sinh thái AI Python; tác vụ PDF nặng có thể tranh chấp tài nguyên với nghiệp vụ chính	<i>Không chọn vì làm liên kết chặt hai loại tải có đặc tính khác nhau</i>
FastAPI xử lý đồng bộ	<i>Dùng trực tiếp thư viện AI Python; kiến trúc đơn giản; phù hợp tìm kiếm ngữ nghĩa cần phản hồi nhanh</i>	Xử lý PDF phải giữ kết nối HTTP lâu; dễ vượt thời gian chờ; cơ chế thử lại phải tự thiết kế	<i>Phù hợp cho yêu cầu AI ngắn, nhưng chưa giải quyết tác vụ PDF dài</i>
FastAPI + RabbitMQ/Celery	<i>API và worker tách trách nhiệm; xử lý PDF nền; hỗ trợ thử lại khi cấu hình; có thể tăng worker độc lập</i>	Tăng thành phần vận hành; phải quản lý hàng đợi, trạng thái tác vụ và tính lặp an toàn	Chọn vì Library74 đồng thời có tìm kiếm cần phản hồi nhanh và xử lý PDF kéo dài

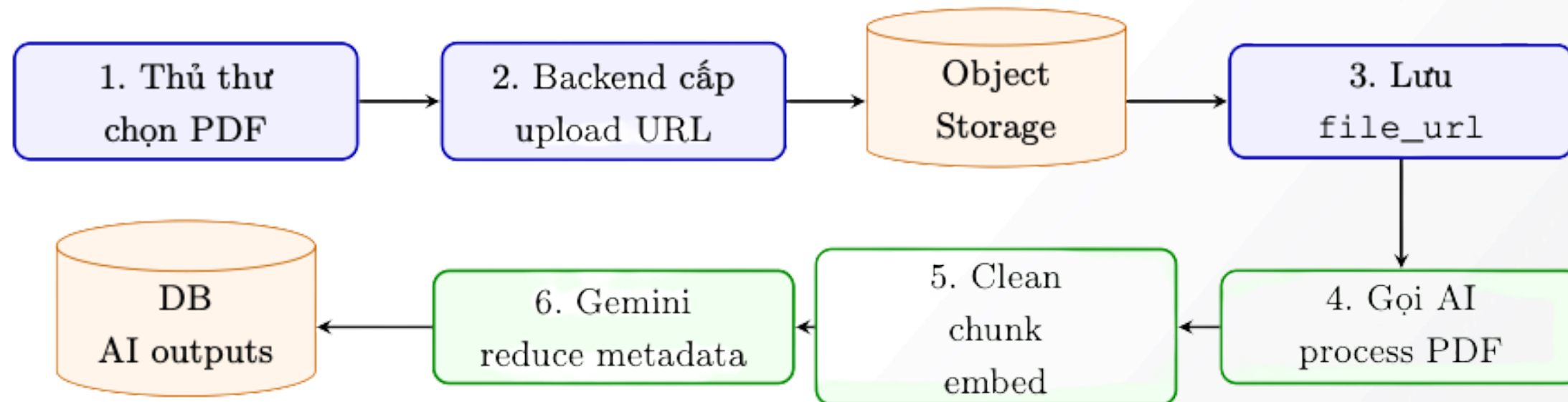
- Đặt AI trong Spring Boot giảm số dịch vụ nhưng làm nghiệp vụ và AI chia sẻ tài nguyên; FastAPI đồng bộ phù hợp yêu cầu ngắn nhưng không phù hợp xử lý PDF kéo dài.
- FastAPI kết hợp RabbitMQ và Celery làm tăng chi phí vận hành, đổi lại API và worker được tách trách nhiệm, tác vụ có thể chạy nền và thử lại.
- => chọn phương án này vì Library74 có đồng thời hai loại tải với vòng đời xử lý khác nhau.

DATABASE: PostgreSQL + pgvector

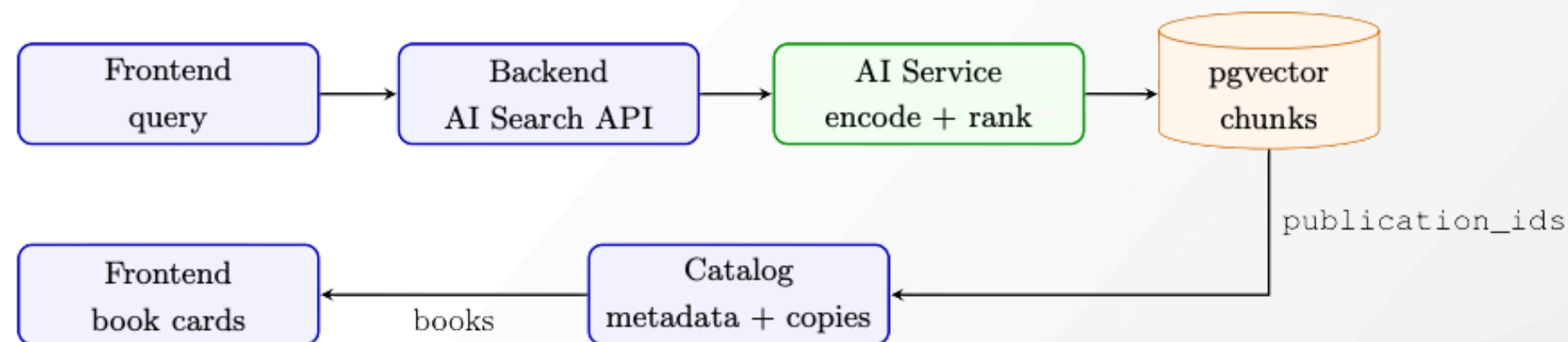
PHƯƠNG ÁN	ƯU ĐIỂM	NHUỘC ĐIỂM	KẾT LUẬN
 MySQL + cơ sở dữ liệu vector riêng	MySQL xử lý nghiệp vụ quan hệ; hệ vector chuyên dụng tối ưu tìm kiếm ngữ nghĩa	Phải đồng bộ ấn phẩm và vector giữa hai hệ	Phù hợp khi cần mở rộng vector độc lập, nhưng làm kiến trúc phức tạp hơn
 MongoDB	Mạnh với dữ liệu tài liệu linh hoạt; có thể bổ sung tìm kiếm vector tùy nền tảng triển khai	Không phải lựa chọn tự nhiên cho chuỗi quan hệ và giao dịch mượn, trả, đặt trước, phí	Phù hợp hệ thống thiên về tài liệu, không phù hợp bằng PostgreSQL với lõi nghiệp vụ Library74
 PostgreSQL + hệ extension	Một database kết hợp giao dịch quan hệ + JSONB + pgvector + pg_trgm + unaccent; hỗ trợ chỉ mục HNSW/GIN và join kết quả AI trực tiếp với ấn phẩm	Cần thiết kế chỉ mục và theo dõi tải khi dữ liệu tăng	Chọn vì PostgreSQL mở rộng từ cơ sở dữ liệu quan hệ thành nền tảng phục vụ đồng thời nghiệp vụ, tìm kiếm văn bản và tìm kiếm ngữ nghĩa

- Chọn PostgreSQL không chỉ là khả năng lưu dữ liệu quan hệ. Điểm khác biệt là hệ extension cho phép mở rộng ngay trên cùng cơ sở dữ liệu.
- Library74 dùng pgvector cho tìm kiếm ngữ nghĩa, pg_trigram và unaccent cho tìm kiếm văn bản gần đúng, JSONB cho metadata linh hoạt, đồng thời vẫn giữ khóa ngoại và giao dịch của nghiệp vụ thư viện.
- Vì tất cả nằm trong PostgreSQL, kết quả vector có thể join trực tiếp với ấn phẩm và tự tuân theo quan hệ dữ liệu, thay vì phải đồng bộ sang một hệ tìm kiếm riêng.
- => Chọn PostgreSQL vì hệ extension giúp một cơ sở dữ liệu đáp ứng cả giao dịch nghiệp vụ, tìm kiếm văn bản và tìm kiếm ngữ nghĩa.

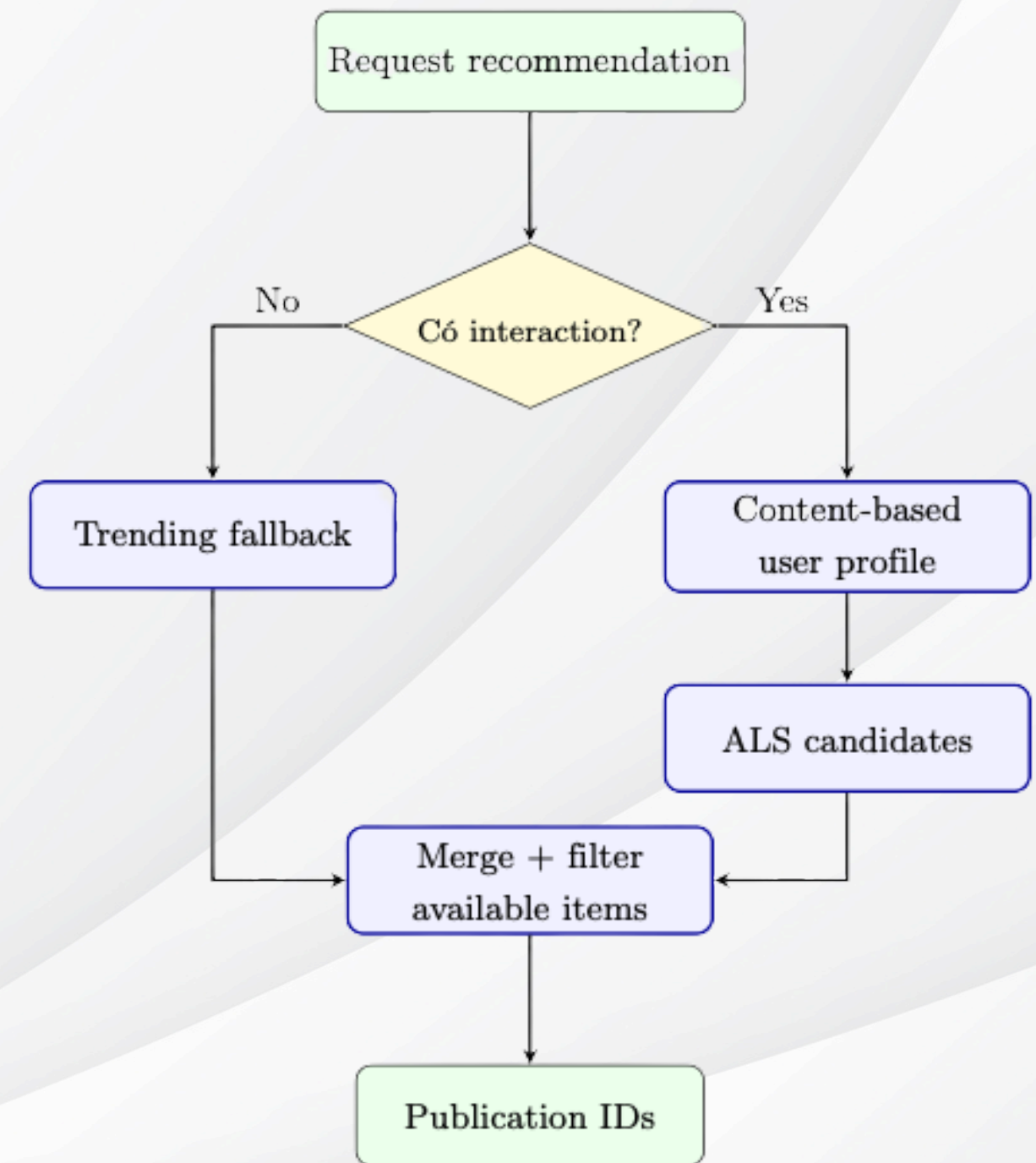
LUỒNG XỬ LÝ AI: 03 MODULE CHÍNH



Module 1 – Luồng xử lý PDF thành tri thức có thể tìm kiếm



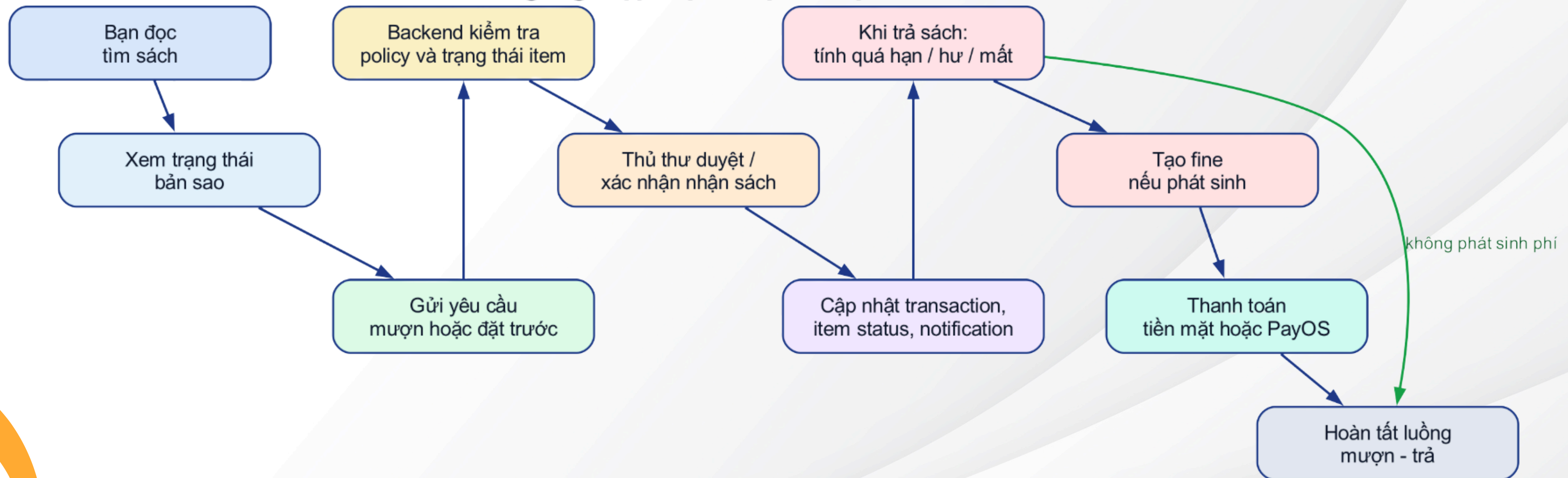
Module 2 – Luồng thực thi semantic search



Module 3: Lược đồ thuật toán phân luồng gợi ý

LUỒNG NGHIỆP VỤ TIÊU BIỂU

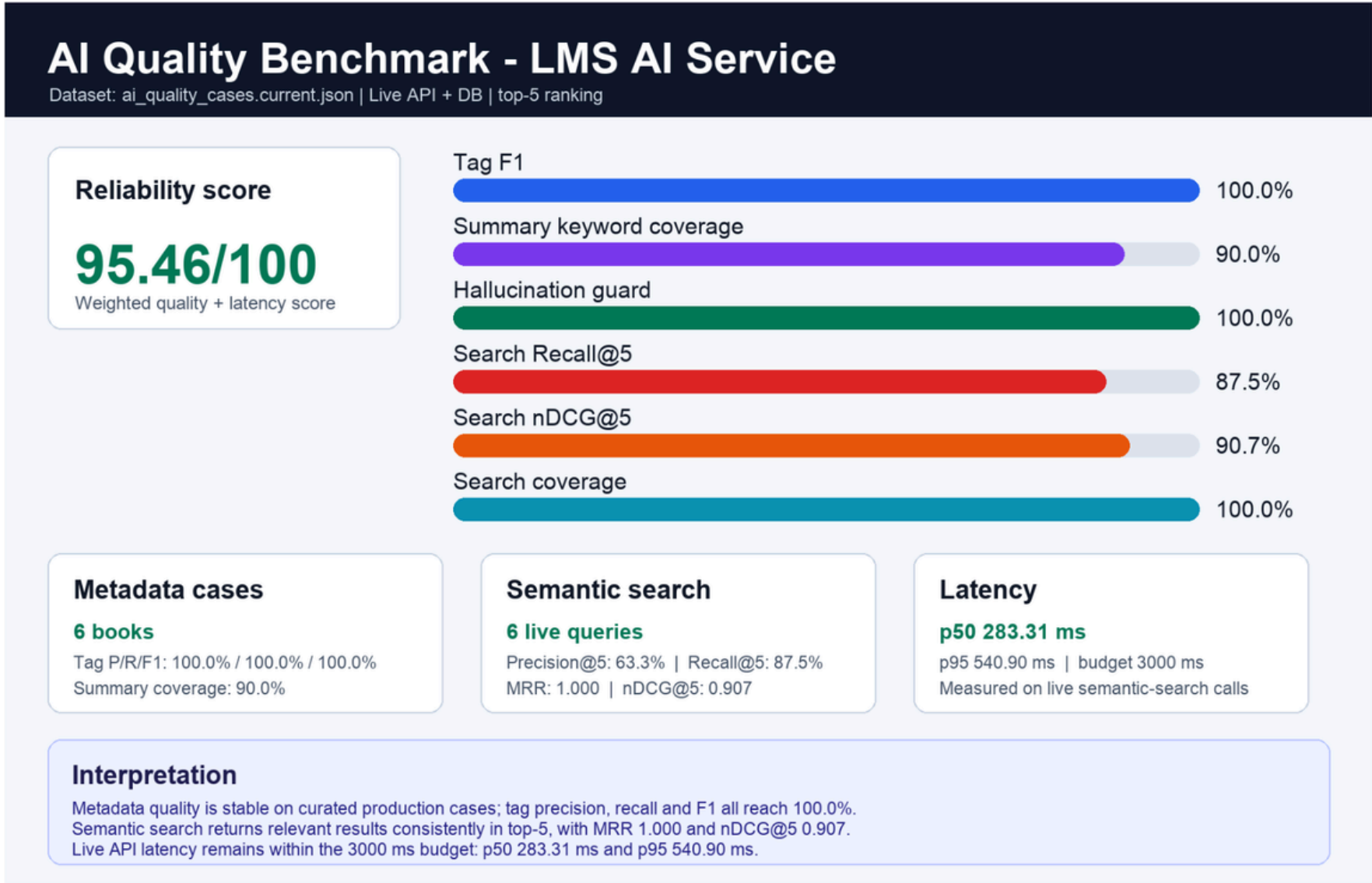
Luồng nghiệp vụ mượn - đặt trước - trả sách



KIỂM THỬ HỆ THỐNG

TẦNG KIỂM THỬ	CÔNG CỤ	KẾT QUẢ	Ý NGHĨA
Frontend unit/contract	<i>Jest, Testing Library</i>	12 suites, 62/62 pass	<i>Giao diện, context đăng nhập/ngôn ngữ/theme/upload và API service hoạt động đúng; giảm rủi ro lỗi UI lan rộng khi đổi code</i>
Backend unit/integration	<i>JUnit, Mockito, MockMvc, Testcontainers</i>	23 classes, 79/79 pass	<i>Nghịệp vụ lõi như auth, catalog, mượn/trả, đặt trước, phí phạt và AI gateway đúng logic; SQL được kiểm với PostgreSQL thật</i>
AI contract/regression	<i>pytest</i>	72 pass, 3 skip	<i>AI pipeline không phải hộp đen: PDF, chunking, LLM guard, SQL contract, recommender, callback và live API đều có kiểm chứng</i>
Production smoke	<i>Python/curl</i>	8/8 pass	<i>Bản triển khai thật trên library74.uk phản hồi đúng ở các luồng quan trọng: homepage, catalog, stats, semantic search, login guard, admin guard</i>
Security/cache headers	<i>curl/header check</i>	Pass	<i>Lớp public có bảo vệ cơ bản: HTTPS/security headers đầy đủ, endpoint nhạy cảm bị chặn, cache static/HTML được cấu hình đúng</i>
Load probe	<i>Python load script</i>	0% lỗi ở 50/100/200 VUs	<i>Hệ thống không crash dưới tải đọc có kiểm soát; throughput bảo hòa nhưng vẫn duy trì error rate 0.00%, cần scale/tối ưu nếu tải lớn hơn</i>

- Kiểm thử được chia nhiều tầng: frontend, backend, AI, integration và production smoke.
- Unit và contract test giúp bắt lỗi nhanh ở từng phần.
- Backend integration test dùng Testcontainers với PostgreSQL thật để giảm sai lệch khi kiểm SQL.
- Production smoke test chạy trên domain library74.uk để kiểm luồng thật sau khi deploy.
- **Hệ thống không chỉ được demo, mà có kiểm thử ở các tầng quan trọng để chứng minh độ ổn định.**



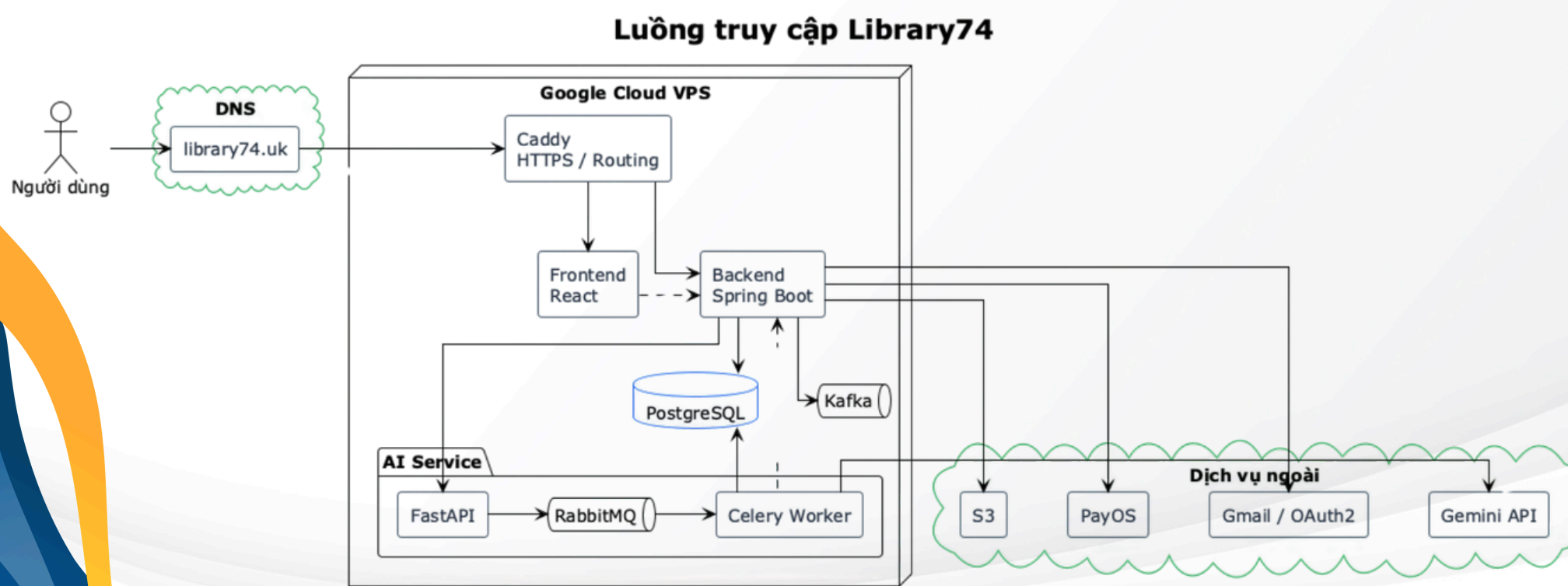
Hình 7.5: Kết quả benchmark định lượng chất lượng AI

AI được đánh giá bằng bộ dữ liệu hiện tại trên production gồm 6 chủ đề: môi trường, hóa học xanh, dầu khí, đại số tuyến tính, xác suất và polymer

Khía cạnh đánh giá	Kết quả định lượng & Chỉ số
Tổng quan định lượng	Điểm tin cậy tổng hợp đạt 95.46/100 trên tập dữ liệu thực tế.
Tìm kiếm ngữ nghĩa	- MRR = 1.000: Kết quả đúng luôn xuất hiện ở vị trí đầu tiên. - Recall@5 = 87.5% & nDCG@5 = 0.907 (Độ phủ Top 5 cực tốt). - Ghi nhận: Có hiện tượng nhiễu nhẹ ở các truy vấn ngách sâu.
Đúc kết tri thức	- Tóm tắt: Độ phủ từ khóa 90.0%, vượt 100% rào chắn chống ảo giác. - Gán nhãn: F1-Score = 100% (Nhất quán tuyệt đối với dữ liệu chuẩn).
Tốc độ phản hồi	Độ trễ p95 = 540.90ms (Vượt xa mức ngân sách giới hạn 3000ms).
Hệ thống gợi ý	Đang vận hành an toàn qua logic dự phòng trong giai đoạn khởi động nguội.

TRIỂN KHAI PRODUCTION VÀ BẢO MẬT

TẦNG KIỂM THỬ	CÔNG CỤ	Ý NGHĨA
Truy cập web	<i>DNS -> Caddy -> Frontend/Backend -> PostgreSQL</i>	<i>Người dùng vào một domain duy nhất, backend xử lý nghiệp vụ và dữ liệu</i>
AI xử lý nền	<i>Backend -> RabbitMQ -> Celery -> pgvector/Gemini -> callback</i>	<i>Tác vụ PDF/AI không làm chậm request chính</i>
Bảo mật vận hành	<i>HTTPS + Docker network + JWT/RBAC + HMAC</i>	<i>Chỉ public 80/443, service nội bộ không lộ trực tiếp</i>



- Hệ thống đã deploy production trên domain library74.uk, không chỉ chạy local.
- Caddy là cửa vào public, nhận HTTPS và điều hướng request đến frontend hoặc backend.
- Frontend gọi API qua Caddy đến Spring Boot; backend xử lý nghiệp vụ rồi đọc ghi PostgreSQL.
- AI xử lý nền qua RabbitMQ/Celery và callback về backend khi hoàn tất.
- Các service nội bộ như PostgreSQL, RabbitMQ, Kafka, backend và AI service không public trực tiếp ra Internet.
- **Phần triển khai đã được kiểm chứng bằng production smoke test 8/8 case pass trên domain thật.**

KẾT QUẢ – GIỚI HẠN – HƯỚNG PHÁT TRIỂN

Hệ thống đã giải quyết đồng thời ba nhóm vấn đề quan trọng: giúp bạn đọc tìm đúng tài liệu nhanh hơn, giúp thủ thư vận hành nghiệp vụ minh bạch hơn, và giúp nhà trường có một nền tảng dữ liệu có thể mở rộng cho chuyển đổi số thư viện.

KẾT QUẢ ĐẠT ĐƯỢC	GIỚI HẠN HIỆN TẠI	HƯỚNG PHÁT TRIỂN
Hoàn thành đầy đủ hệ thống nhiều vai trò	Kiểm thử tải mới dừng ở mức thăm dò có kiểm soát với 200 người dùng ảo hoạt động đồng thời. Hệ thống cần được kiểm chứng tải dài hạn, đồng thời trải qua các giai đoạn cao điểm (mùa thi, đầu học kỳ).	<i>Stress test tìm điểm gãy phần cứng và kiểm thử tải liên tục giả lập chu kỳ mùa thi và đầu học kỳ.</i>
Có nghiệp vụ thư viện đầy đủ	Hệ thống gợi ý chưa có nhiều dữ liệu hành vi thật	<i>Thu thập implicit feedback dài hạn</i>
AI search/ETL hoạt động và có benchmark	OCR hiện là fallback cơ bản; PDF scan mờ/layout phức tạp còn khó xử lý	Nâng cấp OCR và bổ sung layout analysis nâng cao
Triển khai vận hành thực tế, thiết lập các tiêu đề bảo mật, vượt qua kiểm thử nhanh; đã có hệ thống tự động kiểm tra trạng thái hoạt động, trình quản lý giám sát ứng dụng, các công cụ thu thập và cảnh báo dữ liệu và cảnh báo qua ứng dụng Telegram.	Dashboard/alert mới ở mức vận hành cơ bản	Mở rộng dashboard theo SLA, thêm runbook xử lý sự cố và escalation policy

KẾT LUẬN

Dự án “Hệ thống Quản lý Thư viện Trực tuyến tích hợp AI”

- Sản phẩm End-to-End: Hoàn thiện FE, BE, AI Service, CSDL, Queue; triển khai thực tế trên hạ tầng Public.
- Bao phủ nghiệp vụ: Giải quyết trọn vẹn mượn–trả, đặt trước, phí phạt, thông báo realtime và Audit Log.
- Nâng cao trải nghiệm Bạn đọc: Tìm sách nhanh nhờ hiển thị trực thì bản sao, AI Summary và gợi ý tài liệu.
- Tối ưu cho Thủ thư: Gom tác vụ vận hành vào một Dashboard duy nhất, xóa bỏ ghi chú thủ công rời rạc.
- Nâng cao năng lực Quản trị: Phân quyền chặt chẽ, quản lý người dùng tập trung và cấu hình chính sách hệ thống.
- Tích hợp AI đúng vai trò: AI làm trợ lý (Đọc PDF, trích xuất siêu dữ liệu, Semantic Search); Backend kiểm soát nghiệp vụ chính.
- Kỹ thuật nghiêm túc: Kiến trúc tách lớp, giao tiếp bất đồng bộ qua Message Queue và kiểm thử tích hợp chặt chẽ.

DEMO

Tài khoản demo (Link truy cập: library74.uk)			
	Role	Tài khoản	Mật khẩu
User	User1	user1@hcmut.edu.vn	123456
	User2	user2@hcmut.edu.vn	123456
	User3	user3@hcmut.edu.vn	123456
Librarian	Librarian1	librarian1@hcmut.edu.vn	123456
	Librarian2	librarian2@hcmut.edu.vn	123456
	Librarian3	librarian3@hcmut.edu.vn	123456
Admin	Admin	admin@hcmut.edu.vn	admin



ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA

THANK YOU FOR YOUR LISTENING



Library74

NEXT-GEN DISCOVERY

